

Sistemas Operativos

2° Parcial 1C2025 – TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1)

- a) Falso. La segmentación paginada elimina la fragmentación externa, ya que divide cada segmento en páginas de tamaño fijo.
- b) Falso. Aumentar el grado de multiprogramación empeora el thrashing, ya que más procesos compiten por la misma cantidad de marcos. Para solucionarlo, se debe reducir la multiprogramación o aplicar políticas de asignación dinámica de marcos, como el modelo de conjunto de trabajo.

2) La principal ventaja de la paginación invertida es que reduce significativamente el consumo de memoria para las tablas de páginas, ya que utiliza una única tabla global, con una entrada por cada marco físico, sin importar cuántos procesos estén activos.

Sin una función de hash, la traducción de direcciones virtuales requiere buscar línea por línea en la tabla invertida para encontrar una entrada que coincida con el par (*PID, número de página*), lo cual implica un tiempo de búsqueda lineal, haciendo la traducción muy ineficiente.

3) Existen dos tipos de tabla: una global, donde se incluye información sobre los archivos abiertos por cualquier proceso y campos generales (como el contador de aperturas e información del archivo en memoria) y a su vez, una tabla por proceso, donde se guarda un puntero a la tabla global y campos por cada proceso (como el puntero de lectura y el modo de apertura).

Cuando un archivo se abre por primera vez, es escrito en la tabla global y en la del proceso. Cuando el mismo archivo se abre por otro proceso, se escribe sólo en la del proceso y se actualiza el contador de aperturas en la global.

4) La protección de archivos tipo UFS, permite asignar permisos a los archivos para el propietario del archivo, para el grupo asociado al archivo y para el resto de los usuario y grupos. Los permisos que se pueden asignar son de lectura, escritura y ejecución. La principal desventaja es que la asignación es limitada. En caso que se necesite asignar permisos diferentes a varios usuario o grupo, no es posible hacerlo.

Para resolver este problema pueden usarse las Listas de Control de Acceso (ACL). Donde se permite, con mayor granularidad, agregar los permisos que sean necesarios para diferentes usuarios o grupos.

- 5) Los tres métodos pueden sufrir fragmentación interna ya que es independiente del método de asignación y depende del tamaño de los archivos. El método de asignación Contiguo es el único que puede sufrir fragmentación externa debido a que los bloques de un archivo deben estar todos contiguos y quizás existan bloques suficientes en el sistema de archivos, pero no se encuentren contiguos para alojar un archivo.

Para el acceso directo, los métodos más adecuados son el contiguo y el indexado. Para el contiguo, conociendo el primer bloque del archivo, será posible calcular cuál es el bloque buscado ya que estos están contiguos, por lo que no es necesario leer los bloques anteriores al buscado. En el caso del indexado, existe una tabla que contiene punteros a los bloques del archivo por lo que a través de la tabla puede accederse al bloque de manera directa.

Práctica

1) Para hacer los cálculos es necesario conocer el tamaño del puntero que puede calcularse porque en el bloque 5 se muestran 4 punteros. El TB/TP = 4. Si el resultado de dividir el tamaño del bloque que es de 32 bytes por el Tamaño del puntero, es 4, entonces el tamaño del puntero es 8 bytes -> $32 / 8 = 4$

a)

-El tamaño máximo teórico del sistema de archivos:

Cantidad de bloques que pueden ser apuntados * Tamaño del bloque

$2^{\text{Tamaño del Puntero en Bits}} * \text{Tamaño del Bloque en Bytes} = 2^{64} * 2^5 = 2^{69}$ bytes (512 Exabytes)

-El tamaño máximo teórico de un archivo es la suma de:

Tamaño del bloque * (Cantidad de PD + (Tamaño del Bloque / Tamaño del Puntero) + (Tamaño del Bloque / Tamaño del Puntero)²)

$32 * (3 + 4 + 4^2) = 32 * 23 = 736$ bytes

b) Los bloques que forman parte del archivo son 3, 6, 8, 7 y 4, en este orden.

La tabla FAT quedaría:

0	-
1	-
2	-
3	6
4	FIN
5	-
6	8
7	4
8	7
9	-

2) a) El algoritmo usado es clock mejorado. Esto se puede concluir por la existencia de un puntero, en conjunto con la afirmación "Para el reemplazo de la página 3, el puntero dió tres vueltas completas." (si fuera Clock, solo podría dar 2 vueltas como máximo).

b) Valores:

	Lectura pagina 9	Lectura pagina 1
-> 3 => P=1, U=1, M=1	9 => P=1, U=1, M=0	9 => P=1, U=1, M=0
5 => P=1, U=1, M=1	-> 5 => P=1, U=0, M=1	1 => P=1, U=1, M=0
8 => P=1, U=1, M=1	8 => P=1, U=0, M=1	-> 8 => P=1, U=0, M=1

c) Se produjeron dos fallos de página, por lo cual son dos lecturas de disco. Las dos páginas reemplazadas estaban modificadas, así que tenemos también dos escrituras a disco.

d) Debería ser accedida previamente la página 8, para activar su bit de uso.

3) Se debe considerar que los cerrojos son obligatorios, de manera que el proceso 4 no podrá operar sobre el archivo si hay un cerrojo sobre el mismo, y dado que son compartidos, varios procesos podrán leer al mismo tiempo, pero solo uno podrá escribir a la vez.

Nota: Esta solución considera que las syscalls sobre archivos no son bloqueantes.

- El Proceso 1 ejecuta primero, imprimiendo 'A' y volviendo a ready. Lista de bloqueados: vacía
- El proceso 2 ejecuta, bloqueando en "lock" por querer tomar un lock para escritura. No imprime nada. Lista de bloqueados: P2
- El proceso 3 ejecuta, pudiendo tomar el lock aún cuando el proceso 1 no lo ha liberado, por lo tanto imprime 'A', volviendo a Ready. Lista de bloqueados: P2
- El proceso 4 ejecuta, sin tomar lock, intenta leer pero la syscall falla por tener el archivo un lock obligatorio. Imprime '-'. Lista de bloqueados: P2
- El proceso 1 libera el lock
- El proceso 3 libera el lock
- El proceso 2 logra adquirir el lock, modifica el archivo en imprime 'B'.

Impresiones finales: P1:A, P3: A, P4: '-', P2: B